



**Escuela Superior
de Ingeniería y Tecnología**
Universidad de La Laguna

Informe de la Práctica 5 de DAA Curso 2025-2026

*Multi-Source
Capacitated Facility Location Problem
with Customer Incompatibilities
(MS-CFLP-CI)*

Cristhian Cruz Delgado

La Laguna, 28 de abril de 2026



Figura 1: MS-CFLP-CI

La Figura 1 muestra una imagen creada por Nano Banana, tratando de recrear el MS-CFLP-CI abordado en esta práctica.

Agradecimientos

Agradezco a María Belén Melián Batista por proponer esta práctica y a mi familia y amigos por su apoyo incondicional.

Licencia

© Esta obra está bajo una licencia de Creative Commons Reconocimiento-NoComercial 4.0 Internacional.

Resumen

En este informe se aborda la resolución del problema MS-CFLP-CI mediante la implementación de distintos enfoques algorítmicos. El objetivo es explorar algoritmos aproximados capaces de obtener buenas soluciones en instancias donde hallar el óptimo resulta complejo.

MS-CFLP-CI, Waste collection, GRASP, LS, VNS, Reinforcement Learning.

Índice general

1. Introducción	1
1.1. Contexto	1
1.1.1. Objetivos	1
1.2. Motivación	2
2. Multi-Source Capacitated Facility Location Problem with Customer Incompatibilities (MS-CFLP-CI)	3
2.1. Descripción del problema	3
3. Algoritmos	9
3.1. Instancias e implementación	9
3.1.1. Representación de la solución	9
3.2. Algoritmo voraz	10
3.3. Algoritmo GRASP: Fase constructiva	12
3.4. Búsquedas locales	12
4. Experimentos y resultados computacionales	14
4.1. Algoritmo Voraz para MS-CFLP-CI	14
4.2. GRASP para MS-CFLP-CI	15
4.3. GVNS-RL para MS-CFLP-CI	16
4.4. Análisis comparativo entre algoritmos	17
4.4.1. Teórico	17
4.4.2. Práctico	17
5. Conclusiones y trabajo futuro	19
Bibliografía	20

Índice de Figuras

1.	MS-CFLP-CI	2
2.1.	Representación de la instancia del problema.	4
2.2.	Esquema de asignación y flujos entre instalaciones y clientes.	5

Índice de Tablas

2.1.	Parámetros de la instancia de ejemplo ($n = 3, m = 2$)	4
2.2.	Matriz de costes unitarios de transporte (c_{ij})	4
2.3.	Conjunto de pares incompatibles (I_{pairs})	4
2.4.	Solución de Apertura de Instalaciones	5
2.5.	Detalle de Asignación de Clientes	5
2.6.	Verificación de Restricciones	5
2.7.	Desglose Detallado de Costes Variables de Transporte	6
2.8.	Cálculo del Coste Total de la Solución	6
3.1.	Resultados de CPLEX y Gurobi	10
4.1.	Resultados del Algoritmo Voraz. Se detalla el número de fuentes abiertas e incompatibilidades.	14
4.2.	Resultados de GRASP con diferentes tamaños de LRC para el problema MS-CFLP-CI.	15
4.3.	Resultados de GVNS aplicando diferentes estructuras de vecindad (k_{max}).	16

Capítulo 1

Introducción

1.1. Contexto

Para la práctica 5 de la asignatura de Diseño y Análisis de Algoritmo se propone resolver el problema MS-CFLP-CI. Para ello, se propone al alumnado generar código en C++ para poder obtener mediante distintas metodologías una solución al problema.

1.1.1. Objetivos

Durante el desarrollo de esta práctica, se han cumplido los objetivos listados a continuación:

- Modelizar la instancia, solución y representación del MS-CFLP-CI.
- Resolver el problema mediante un algoritmo Greedy.
- Resolver el problema mediante un algoritmo GRASP (construction phase + 1 local search phase).
 - 1 búsqueda local para reinserción.
 - 2 búsquedas locales para intercambio.
 - 1 búsqueda local para reasignación.
- Resolver el problema mediante un algoritmo GVNS (shaking + 1 local search).
 - 1 búsqueda local VND.
 - 1 búsqueda local RVND.
 - 1 búsqueda local RL.

1.2. Motivación

El propósito de la práctica es trabajar con distintos tipos de algoritmos, en especial trabajar con algoritmos aproximados. Dado el caso, ya que en el problema nos enfrentamos a instancias para las que es difícil encontrar la solución óptima, los algoritmos aproximados serán de especial valor para encontrar soluciones tan buenas como la proporción del tamaño de la instancia lo permita.

Capítulo 2

Multi-Source Capacitated Facility Location Problem with Customer Incompatibilities (MS-CFLP-CI)

2.1. Descripción del problema

En esta práctica de Diseño y Análisis de Algoritmos trabajaremos en una variante del problema clásico de localización de instalaciones capacitadas (*Capacitated Facility Location Problem* - CFLP), denominado *Multi-source Capacitated Facility Location Problem with Customer Incompatibilities* (MS-CFLP-CI) (Gjergji et al., 2025). Esta variante es especialmente relevante en logística y recogida de residuos, donde existen multitud de restricciones de competencia comercial o seguridad.

Se nos presenta un escenario con los siguientes componentes:

- **Clientes (I):** Un conjunto de n clientes, donde cada cliente i tiene una demanda específica d_i .
- **Instalaciones (J):** Un conjunto de m ubicaciones potenciales para abrir instalaciones. Cada una tiene un coste fijo de apertura f_j y una capacidad máxima de suministro s_j .
- **Costos de Transporte:** Un coste c_{ij} por cada unidad de demanda enviada desde la instalación j al cliente i .
- **Restricciones de Incompatibilidad (C):** Un conjunto de pares de clientes $\langle i_1, i_2 \rangle$ que **no pueden** ser abastecidos por la misma instalación

bajo ninguna circunstancia.

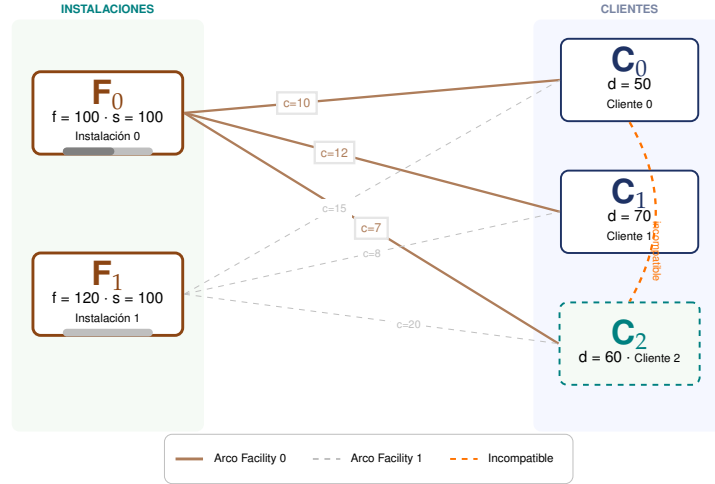


Figura 2.1: Representación de la instancia del problema.

Tabla 2.1: Parámetros de la instancia de ejemplo ($n = 3, m = 2$)

Instalaciones (Facilities)			Clientes (Customers)		
ID	Coste Fijo (f_j)	Capacidad (s_j)	ID	Demanda (d_i)	Incompatibilidad
0	100	100	0	50	con Cliente 2
1	120	100	1	70	-
			2	60	con Cliente 0

Tabla 2.2: Matriz de costes unitarios de transporte (c_{ij})

Cliente \ Instalación	Facility 0	Facility 1
0	10	15
1	12	8
2	7	20

Tabla 2.3: Conjunto de pares incompatibles (I_{pairs})

Cliente i_1	Cliente i_2
0	2

Se deben resolver dos problemas de decisión simultáneamente para minimizar el coste total (apertura + transporte):

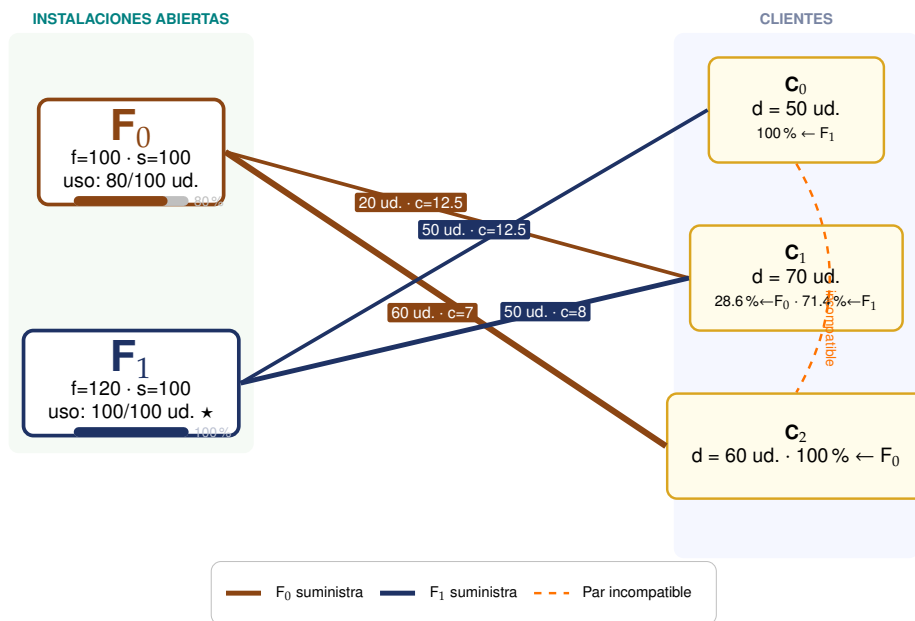


Figura 2.2: Esquema de asignación y flujos entre instalaciones y clientes.

Tabla 2.4: Solución de Apertura de Instalaciones

Instalación	Estado	Coste Fijo
Instalación 0	Abierta	100
Instalación 1	Abierta	120
Total Costo Fijo		220

Tabla 2.5: Detalle de Asignación de Clientes

Cliente	Instalación	Demanda Asignada	Porcentaje (%)
Cliente 0	Instalación 1	50.00	100.0 %
Cliente 1	Instalación 0	20.00	28.6 %
	Instalación 1	50.00	71.4 %
Cliente 2	Instalación 0	60.00	100.0 %

Tabla 2.6: Verificación de Restricciones

Instalación	Carga Total	Capacidad	Estado
Instalación 0	80.00 (20+60)	100	✓OK
Instalación 1	100.00 (50+50)	100	✓Límite
Incompatibilidad	Cliente 0 y 2 en plantas distintas		✓OK

Tabla 2.7: Desglose Detallado de Costes Variables de Transporte

Origen	Destino	Cantidad (q)	Coste Unitario (c)	Subtotal
Instalación 2	Cliente 0	50.00	15.00	750.00
Instalación 1	Cliente 1	20.00	12.00	240.00
Instalación 2	Cliente 1	50.00	8.00	400.00
Instalación 1	Cliente 2	60.00	7.00	420.00
Total Costes Variables				1,810.00

Tabla 2.8: Cálculo del Coste Total de la Solución

Concepto	Coste
Costos Fijos (Apertura f_j)	220.00
Costos Variables (Transporte u_{ij})	1,810.00
Costo Total (Función Objetivo)	2,030.00

1. **Localización de instalaciones:** Determinar qué subconjunto de instalaciones $J' \subseteq J$ debe abrirse.
2. **Asignación de flujo (Multi-source):** Determinar cuánto suministro envía cada instalación abierta a cada cliente. A diferencia de otros modelos, un cliente puede recibir productos de múltiples instalaciones simultáneamente para satisfacer su demanda total.

Para los estudiantes de algoritmos, este problema es de especial interés porque:

- El problema de localización base (CFLP) es **NP-hard**.
- En la variante estándar, una vez decididas qué instalaciones abrir, la asignación óptima es un problema de transporte fácil (polinomial).
- Sin embargo, en el **MS-CFLP-CI**, el simple hecho de asignar clientes a instalaciones abiertas se vuelve **NP-hard** debido a las restricciones de incompatibilidad, que equivalen en esencia a un problema de coloración de grafos restringido.

El **Problema de Localización de Instalaciones con Capacidad e Incompatibilidades entre Clientes (MS-CFLP-CI)** es una variante del problema de localización clásico. En este escenario, debemos decidir qué instalaciones abrir (como almacenes o centros de residuos) y cómo asignar a los clientes para satisfacer su demanda al menor coste posible.

Este modelo introduce dos características clave:

1. **Multi-suministro (Multi-source):** Cada cliente puede recibir sus bienes o depositar sus residuos en múltiples instalaciones abiertas simultáneamente para completar su demanda total.
2. **Incompatibilidades:** Existen pares de clientes que, por motivos estratégicos (competencia de mercado) o de seguridad (materiales peligrosos o incompatibles), **no pueden ser atendidos por la misma instalación.**

El **objetivo** del MS-CFLP-CI es minimizar la suma de los costes fijos de apertura de instalaciones y de los costes variables de transporte.

Modelo Matemático

Sean los conjuntos:

- $J = \{1, \dots, m\}$ el conjunto de posibles ubicaciones para las instalaciones.
- $I = \{1, \dots, n\}$ el conjunto de clientes.
- $\mathcal{I}$ el conjunto de pares de clientes incompatibles $\langle i_1, i_2 \rangle$.

Y los parámetros:

- f_j : Coste fijo de abrir la instalación $j \in J$.
- s_j : Capacidad máxima de la instalación $j \in J$.
- d_i : Demanda total del cliente $i \in I$.
- c_{ij} : Coste unitario de transporte desde la instalación j al cliente i .

Variables de Decisión

- $y_j \in \{0, 1\}$: Variable binaria igual a 1 si la instalación j se abre, 0 en caso contrario.
- $x_{ij} \in [0, 1]$: fracción de la demanda del cliente i satisfecha por la instalación j .
- $w_{ij} \in \{0, 1\}$: variable binaria que vale 1 si el cliente i es servido (aunque sea parcialmente) por la instalación j , y 0 si no lo es.

Formulación

$$\text{mín } Z = \sum_{j \in J} f_j y_j + \sum_{i \in I} \sum_{j \in J} (c_{ij} \cdot d_i) x_{ij} \quad (2.1)$$

Sujeto a:

- **Satisfacción de demanda:** Toda la demanda de cada cliente debe ser cubierta.

$$\sum_{j \in J} x_{ij} = 1, \quad \forall i \in I \quad (2.2)$$

- **Restricción de capacidad:** El suministro total de una instalación no puede superar su capacidad, y solo puede suministrar si está abierta.

$$\sum_{i \in I} d_i x_{ij} \leq s_j y_j, \quad \forall j \in J \quad (2.3)$$

- **Vínculo de suministro:** Si hay flujo hacia un cliente, la variable de servicio w_{ij} debe ser 1.

$$x_{ij} \leq w_{ij}, \quad \forall i \in I, j \in J \quad (2.4)$$

- **Incompatibilidad de clientes:** Dos clientes incompatibles no pueden compartir instalación.

$$w_{i_1 j} + w_{i_2 j} \leq 1, \quad \forall \langle i_1, i_2 \rangle \in \mathcal{I}, \forall j \in J \quad (2.5)$$

- **Dominios:**

$$y_j \in \{0, 1\}, \quad w_{ij} \in \{0, 1\}, \quad 0 \leq x_{ij} \leq 1 \quad (2.6)$$

Capítulo 3

Algoritmos

El proyecto consiste en diseñar e implementar algoritmos constructivos y basados en búsquedas locales para resolver un conjunto de instancias del MS-CFLP-CI, capaces de encontrar soluciones de alta calidad en tiempos de computación razonables. Se deberá validar que la solución no viole las capacidades ni las restricciones de incompatibilidad.

3.1. Instancias e implementación

Las instancias del problema se suministrarán en un fichero de texto con el formato mostrado en las instancias proporcionadas.

Mejores soluciones conocidas para las instancias wlp01 a wlp08

La Tabla 3.1 muestra los mejores valores conocidos para las instancias wlp01 a wlp08. Los valores en negrita indican que tanto Cplex como Gurobi alcanzaron la solución óptima.

3.1.1. Representación de la solución

Determinar cómo representar las soluciones de manera eficiente, de tal manera que se puedan realizar sobre ellas movimientos de reinserción e intercambio de clientes y apertura, cierre e intercambio de falicilies.

Tabla 3.1: Resultados de CPLEX y Gurobi

Instancia	m	n	CPLEX	Gurobi
wlp01	50	115	28716	28716
wlp02	100	253	52952	52952
wlp03	150	345	64296	64296
wlp04	200	479	84633	84633
wlp05	250	601	107323	103857
wlp06	300	705	115295	111654
wlp07	400	1012	170100	162277
wlp08	500	1277	–	187938

3.2. Algoritmo voraz

Este algoritmo ha sido propuesto en el trabajo de Gjergji et al., 2025.

Algoritmo 1: Algoritmo voraz para el MS-CFLP-CI

Entrada: Conjunto de instalaciones F , conjunto de clientes I , demandas d_i , capacidades s_j , costes de apertura f_j , costes de asignación c_{ij} , parámetro de holgura $k = 5$

Salida: Conjunto de instalaciones abiertas F_{open} , plan de asignación x_{ij}

// Fase 1: Selección de Instalaciones

- 1 Ordenar F en orden ascendente según f_j ;
- 2 $D_{total} \leftarrow \sum_{i \in I} d_i$;
- 3 $CapacidadAcumulada \leftarrow 0$;
- 4 $F_{open} \leftarrow \emptyset$;
- 5 **foreach** instalación $j \in F$ **do**
- 6 **if** $CapacidadAcumulada < D_{total}$ **then**
- 7 $F_{open} \leftarrow F_{open} \cup \{j\}$;
- 8 $CapacidadAcumulada \leftarrow CapacidadAcumulada + s_j$;
- 9 // Añadir holgura por incompatibilidad
- 10 $F_{extra} \leftarrow$ seleccionar las siguientes k instalaciones de la lista ordenada no presentes en F_{open} ;
- 11 $F_{open} \leftarrow F_{open} \cup F_{extra}$;
- 12 // Fase 2: Asignación de Clientes
- 13 Inicializar capacidades residuales $r_j = s_j$ para todo $j \in F_{open}$;
- 14 Inicializar $clientesEn_j = \emptyset$ para todo $j \in F_{open}$;
- 15 Inicializar $x_{ij} = 0$ para todo $i \in I, j \in F_{open}$;
- 16 **foreach** cliente $i \in I$ **do**
- 17 $L_i \leftarrow$ ordenar F_{open} en orden ascendente según c_{ij} ;
- 18 **while** $d_i > 0$ **do**
- 19 $j \leftarrow$ siguiente instalación en L_i ;
- 20 // Verificar compatibilidad: ningún cliente incompatible con i está en j
- 21 **if** $\nexists \langle i, i' \rangle \in I$ tal que $i' \in clientesEn_j$ **then**
- 22 $q \leftarrow \min(d_i, r_j)$;
- 23 $x_{ij} \leftarrow x_{ij} + \frac{q}{d_i}$ (Asignar q unidades de cliente i a instalación j);
- 24 $d_i \leftarrow d_i - q$;
- 25 $r_j \leftarrow r_j - q$;
- 26 $clientesEn_j \leftarrow clientesEn_j \cup \{i\}$;
- 27 // Calcular el coste de la solución construida, S
- 28 $fixedCost \leftarrow \sum_{j \in F_{open}} f_j$;
- 29 $transportCost \leftarrow \sum_{i \in I} \sum_{j \in F_{open}} c_{ij} \cdot d_i \cdot x_{ij}$;
- 30 $totalCost \leftarrow fixedCost + transportCost$;
- 31 Devolver la solución S y su valor de función objetivo, $f(S)$

3.3. Algoritmo GRASP: Fase constructiva

Diseña e implementa la fase constructiva de GRASP, introduciendo los componentes de aleatoriedad y adaptatividad de dicho algoritmo. El diseño es libre y se valorará el grado de adaptación a las particularidades del problema.

Aquellos estudiantes que tengan dificultades con la definición de un nuevo algoritmo constructivo adaptado al MS-CFLP-CI, pueden optar por incluir algún componente de aleatoriedad al Algoritmo voraz 1.

3.4. Búsquedas locales

Se deben implementar las búsquedas locales voraces derivadas de los siguientes movimientos:

Operadores de entorno para MS-CFLP-CI

Para mejorar la solución de la fase constructiva, se definen los siguientes movimientos:

- **Shift(i, j_1, j_2): (Reinserción)** Reasigna una cantidad de demanda del cliente i de la instalación j_1 a la j_2 . Requiere verificar: $r_{j_2} \geq d_i$ y compatibilidad de i con clientes en j_2 . La cantidad a reasignar puede ser una fracción o la total.
- **Swap-Clientes(i_1, i_2):** Intercambia las asignaciones de dos clientes entre sus respectivas instalaciones. Se debe comprobar factibilidad (capacidad disponible e incompatibilidad con otros clientes). Es útil para liberar capacidad sin alterar el total de demanda servida por planta.
- **Swap-Instalaciones(j_{open}, j_{closed}):** Cierra una instalación activa y abre una inactiva, reasignando todos los clientes de la primera a la segunda (o a otras abiertas).
- **Eliminación de Incompatibilidad:** Identifica una instalación con alto coste de transporte y mueve al cliente que causa más bloqueos por incompatibilidad a otra planta, permitiendo una reasignación masiva más eficiente.

Algoritmo GVNS-RL

Implementar un algoritmo GVNS en el que la perturbación se realiza mediante la reinserción de un cliente de una instalación a otra y la fase de mejora se realiza mediante un VND con Reinforcement Learning.

Capítulo 4

Experimentos y resultados computacionales

4.1. Algoritmo Voraz para MS-CFLP-CI

El diseño del algoritmo voraz consta de dos fases, una para elegir un subconjunto del total de instalaciones, además se ha optado por añadir un parámetro de holgura para asignar más instalaciones para contar con exceso de capacidad que permita gestionar incompatibilidades entre clientes, la otra fase pasa a asignar todos los clientes a las instalaciones ya elegidas. Su implementación ha sido en un primer lugar calcada según el pseudo-código de la página 11, luego se ha ido adaptando a los requerimientos de memoria y tiempo de ejecución para poder mejorar la calidad de la experimentación propuesta.

Algoritmo Voraz (MS-CFLP-CI)						
Instancia	$ J_{open} $	Coste Fijo	Coste Asig.	Coste Total	Incomp.	CPU_Time (s)
wlp01.dzn	34	16980	20024	37004	0	0.01
wlp02.dzn	66	34590	35723	70313	0	0,04
wlp03.dzn	93	46400	40077	86477	0	0.11
⋮	⋮	⋮	⋮	⋮	⋮	⋮
wlp20.dzn						

Tabla 4.1: Resultados del Algoritmo Voraz. Se detalla el número de fuentes abiertas e incompatibilidades.

4.2. GRASP para MS-CFLP-CI

Este algoritmo, al igual que el voraz, está compuesto por dos fases. En primer lugar, una fase constructiva, en la que se aplica el propio algoritmo voraz, pero incluyendo una modificación para añadir aleatoriedad, esto se implementa mediante una LRC. La lista de candidatos puede variar su tamaño y permite añadir aleatoriedad en la construcción manteniendo valores cercanos a lo que el algoritmo voraz puro podía ofrecer. Otro parámetro relevante para este algoritmo GRASP es el número de iteraciones del bucle, aunque primero deberíamos atacar las búsquedas locales. Para este problema se han implementado cuatro búsquedas locales lo que permite modificar la solución construida para poder llegar a nuevas soluciones, y que junto al factor aleatorio están regiones sean distintas de las que nos ofrecía el algoritmo voraz. Ahora sí, se une la fase constructiva junto con una búsqueda local por un número de iteraciones y de ella se obtienen valores con los que hemos podido experimentar en esta práctica. Durante la experimentación el número de iteraciones utilizado es el máximo para encontrar soluciones de interés en el tiempo mínimo.

Algoritmo GRASP (Ajuste y Resultados)								
Instancia	LRC	B. Local	J _{open}	C. Fijo	C. Asig.	C. Total	Incomp.	CPU_Time
wlp01.dzn	2	shift	34	16980	19542	36522	0	208.70
wlp01.dzn	2	swapS	34	16980	19454	36434	0	217.99
wlp01.dzn	2	swapW	34	17500	19454	36594	0	254.72
wlp01.dzn	2	incomp	34	16980	19968	36948	0	197.63
wlp01.dzn	3	shift	34	16980	20280	37260	0	363.80
wlp01.dzn	3	swapS	34	16980	19701	36681	0	361.41
wlp01.dzn	3	swapW	34	17170	21502	38672	0	270.52
wlp01.dzn	3	incomp	34	16890	21690	38670	0	206.87
wlp20.dzn								
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
wlp20.dzn								
Promedio			34	17000	20000	37000	0	250.00

Tabla 4.2: Resultados de GRASP con diferentes tamaños de LRC para el problema MS-CFLP-CI.

4.3. GVNS-RL para MS-CFLP-CI

El diseño del algoritmo GVNS, es el más robusto de los diseñados hasta el momento. Combina todo lo anterior y se añade una perturbación inicial de la solución, por ejemplo, para generar la solución inicial sobre la que se aplica la metaheurística. De manera que, la idea es partir de una solución parcialmente buena, cercana a soluciones mejores para aplicar el algoritmo. Para encontrar las soluciones mencionadas se aplica la perturbación que diferencia a la solución inicial por una distancia. La longitud se mide mediante un parámetro. Este será el primer parámetro que toma el algoritmo y se tendrá en cuenta a la hora de experimentar. En este caso, una única estructura de entornos es considerada en la perturbación para después aplicar la búsqueda local pertinente. Este proceso se repite por un número definido de iteraciones que es un parámetro del algoritmo y que en cualquier caso cuanto más grande mejor.

Algoritmo GVNS (Ajuste y Resultados)								
Instancia	k_{max}	B. Local.	$ J_{open} $	C. Fijo	C. Asig.	C. Total	Incomp.	CPU_Time
wlp01.dzn	10	VND	34	17370	18934	36304	0	316.09
wlp01.dzn	10	RVND	34	17970	15746	33716	0	523.10
wlp01.dzn	20	VND	34	17500	16804	34304	0	817.56
wlp01.dzn	20	RVND	34	18120	13846	31966	0	2795.62
wlp01.dzn	30	VND	34	17280	16463	33743	0	1191.69
wlp01.dzn	30	RVND	34	17800	12562	30332	0	7106.19
wlp20.dzn								
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
wlp20.dzn								
Promedio								

Tabla 4.3: Resultados de GVNS aplicando diferentes estructuras de vecindad (k_{max}).

4.4. Análisis compartativo entre algoritmos

4.4.1. Teórico

Es difícil analizar estos algoritmos, ya que uno no se entiende sin el otro. Cada uno representa la evolución para dar solución a problemas tan grandes como las instancias del problema que estamos modelando. El elemento fundamental que los distingue es el equilibrio entre diversificación e intensificación.

En primer lugar, se parte de un algoritmo greedy, un algoritmo aproximado que no da pie a la diversificación y que sí nos proporciona un óptimo local, pero que está lejos de los valores óptimos de la solución.

Decidimos añadir diversificación incorporando la RCL al procedimiento greedy, y como no estamos ante un óptimo local, podemos alcanzarlo ejecutando una búsqueda local simple como elemento explotativo del algoritmo. El factor aleatorio requiere probar distintas ejecuciones del random greedy para encontrar un buen óptimo local en varias iteraciones. Dado que solo consideramos una estructura de entorno en la mejora, es necesario diseñar algo más sofisticado que este GRASP, que permita refinar los resultados de la heurística.

Finalmente, estamos ante un algoritmo metaheurístico. El GVNS trabaja sobre una solución inicial cualquiera; en la implementación se ha optado por construirla con random greedy y aplicarle después una búsqueda local que la mejora. Para partir de buenas soluciones, que suelen estar más próximas al óptimo local, se aplica un RVND que permite mantener la diversificación. Así, se parte de una solución y, durante varias iteraciones, se exploran diferentes óptimos locales cercanos a la solución inicial para intentar alcanzar el óptimo global, lo que se define como perturbación. En definitiva, se aplica diversificación sobre la solución, mientras que la intensificación se modela con la búsqueda local, que ahora incorpora cuatro estructuras de entorno. Si se desea enfatizar la diversificación, se puede aplicar RVND; si se prioriza la intensificación, VND; y si se busca un equilibrio efectivo entre ambas, es necesario ajustar los parámetros de una búsqueda local basada en aprendizaje por refuerzo.

4.4.2. Práctico

Si bien hemos hecho una comparativa a nivel estructural, lo cierto es que el objetivo de la práctica es hacer uso de las implementaciones. Ya que hemos diseñado estos algoritmos y podemos probarlos, tenemos conclusiones prácticas y experimentales con las que contrastar el planteamiento

teórico.

Vamos a hacer de nuevo un análisis ascendente en lo que a robustez del algoritmo se refiere y por ello comenzamos con el greedy. Resultados obtenidos para este algoritmo, en realidad no son malos pese a que antes no se haya considerado como una opción muy descartable. Obtenemos valores no cercanos a óptimos pero, que en relación a los siguientes algoritmos y en función del tiempo y complejidad de la implementación pues no están tan mal. No obstante, aún nos encontramos lejos del óptimo global y de ninguna manera la solución puede variar.

Nos interesa optar a una mejor solución, y el GRASP en alguno de los casos nos la ha proporcionado, aunque en otros no. La construcción greedy aleatoria es la responsable de que varíen estas soluciones. Lo que diferencia al greedy puro de este GRASP es el tiempo de ejecución. Entonces, por qué se propone este como una mejor opción que el primero, pues porque realmente si se han encontrado mejores soluciones. Ya por esto vale la pena, si se hacen más pruebas se encontrarán mejores todavía. Además, que depende de la búsqueda local elegida, por eso hemos añadido una ejecución de cada una en la tabla de resultados y también hemos distinguido el tamaño de la LRC. Este es el parámetro más importante del algoritmo que determina la diversificación admitida, de hecho si asumimos una RCL de tamaño uno, entonces estamos ante una fase constructiva equivalente a un greedy puro. A modo de conclusión se determina que añadiendo una pequeña diversificación de tamaño dos en la LRC se consiguen mejores resultados obtenidos por la intensificación aplicada a través de la búsqueda local sobre el número de iteraciones que se haya computado el algoritmo.

Finalmente, hemos conseguido diseñar un algoritmo cuyas soluciones si son potencialmente mejores que las anteriores. Ya se puede observar una clara decaída de los valores, pese aún a un valor de tiempo de ejecución superior. No es algo casual, las soluciones también mejoran según la distancia permitida por la permutación aumenta. Por lo que, las conclusiones son claras, un algoritmo metaheurístico como el GVNS que permite visitar distintos óptimos locales a distintas distancias al final lo que permite es visitar todas las soluciones óptimas (locales), y eventualmente la óptima global.

Capítulo 5

Conclusiones y trabajo futuro

En este informe se recogen los resultados de distintas implementaciones en C++ de distintos algoritmos. Este proceso ha destacado por un profundo aprendizaje en conceptos algorítmicos unido a la adquisición de nuevas técnicas o estándares de programación.

Por el momento, se ha desarrollado un criterio sólido para la aplicación de las características básicas de un algoritmo aproximados que son la diversificación e intensificación.

A lo sumo, se ha trabajado en profundidad con los conceptos de espacio de soluciones y solución; con entorno y solución vecina; y con función de evaluación y evaluación del delta. Todo ello ha sido diseñado y programado y se refleja en la implementación, así como se refleja que el impacto de la codificación de la representación de la solución condiciona el cálculo del valor de la función de evaluación, que además determina el tamaño del espacio de soluciones y que influye directamente en la eficiencia de los algoritmos.

Bibliografía

Gjergji, Ida et al. (2025). "Large Neighborhood Search for Capacitated Facility Location with Customer Incompatibilities". En: GECCO '25. NH Malaga Hotel, Malaga, Spain: Association for Computing Machinery, págs. 213-221. ISBN: 9798400714658. DOI: 10.1145/3712256.3726355. URL: <https://doi.org/10.1145/3712256.3726355>.